

AI ASSISTED CODING

Lab Assignment 13

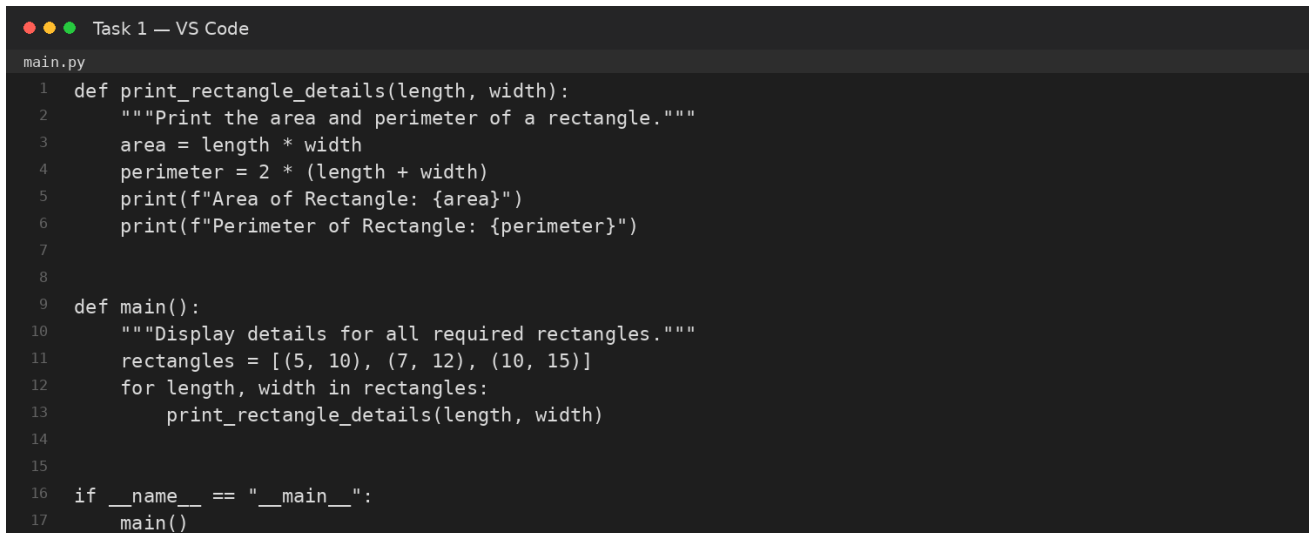
NAME:K.SAI VARDHAN REDDY

ROLL NO:2503A51266

1. Removing Code Duplication

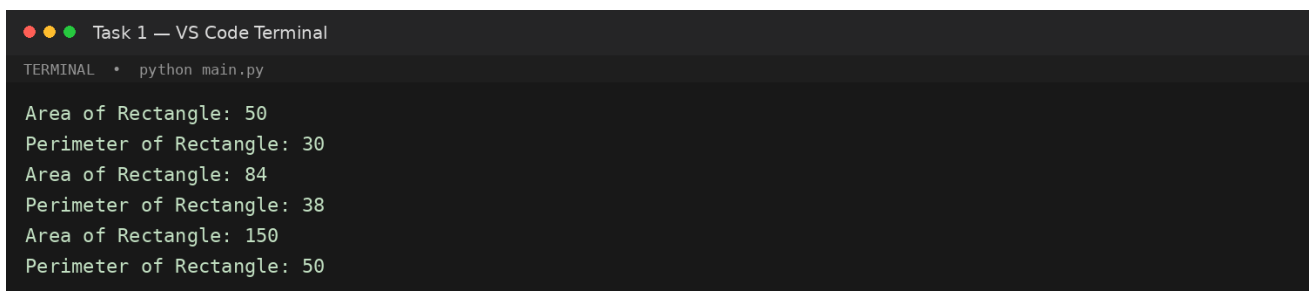
Task requirement: Refactoring: one reusable function replaces three duplicated blocks. [filecite@turnOfile0L45-L60](#)

CODE — VS CODE SCREENSHOT



```
Task 1 — VS Code
main.py
1 def print_rectangle_details(length, width):
2     """Print the area and perimeter of a rectangle."""
3     area = length * width
4     perimeter = 2 * (length + width)
5     print(f"Area of Rectangle: {area}")
6     print(f"Perimeter of Rectangle: {perimeter}")
7
8
9 def main():
10     """Display details for all required rectangles."""
11     rectangles = [(5, 10), (7, 12), (10, 15)]
12     for length, width in rectangles:
13         print_rectangle_details(length, width)
14
15
16 if __name__ == "__main__":
17     main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



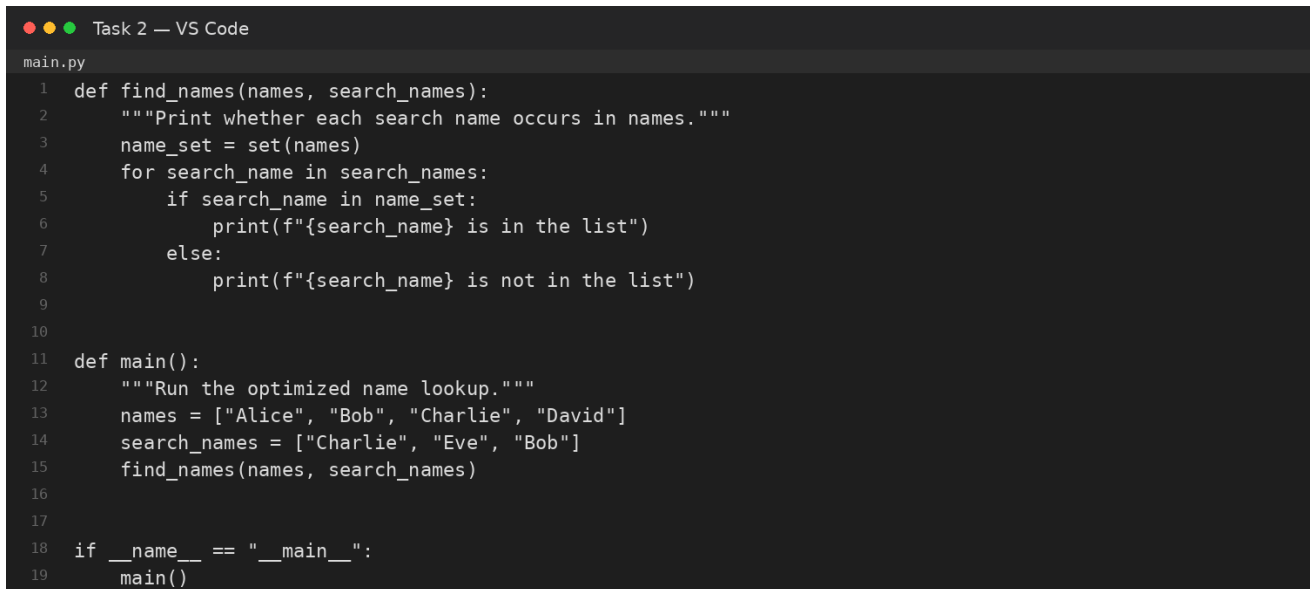
```
Task 1 — VS Code Terminal
TERMINAL • python main.py

Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

2. Optimizing Loops and Conditionals

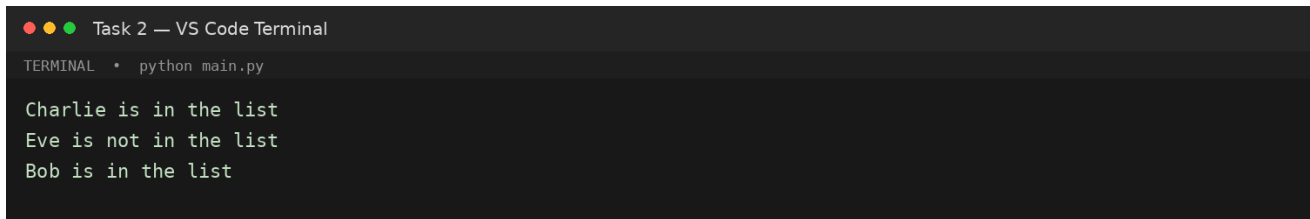
Task requirement: Performance comparison: nested-list search is $O(n \times m)$ in the worst case; set membership is average $O(1)$, improving repeated lookups. [filecite\turn0file0\L61-L83](#)

CODE — VS CODE SCREENSHOT



```
Task 2 — VS Code
main.py
1 def find_names(names, search_names):
2     """Print whether each search name occurs in names."""
3     name_set = set(names)
4     for search_name in search_names:
5         if search_name in name_set:
6             print(f"{search_name} is in the list")
7         else:
8             print(f"{search_name} is not in the list")
9
10
11 def main():
12     """Run the optimized name lookup."""
13     names = ["Alice", "Bob", "Charlie", "David"]
14     search_names = ["Charlie", "Eve", "Bob"]
15     find_names(names, search_names)
16
17
18 if __name__ == "__main__":
19     main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



```
Task 2 — VS Code Terminal
TERMINAL • python main.py

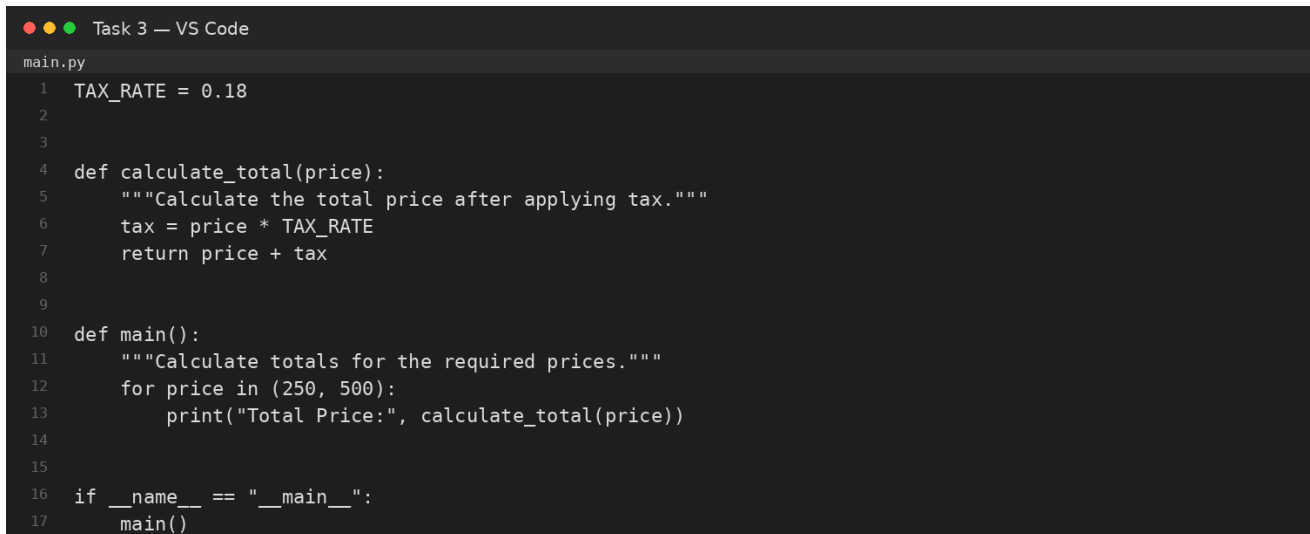
Charlie is in the list
Eve is not in the list
Bob is in the list
```

3. Extracting Reusable Functions

Task requirement: Related calculation logic is isolated in `calculate_total()`, so it can be reused for any price.

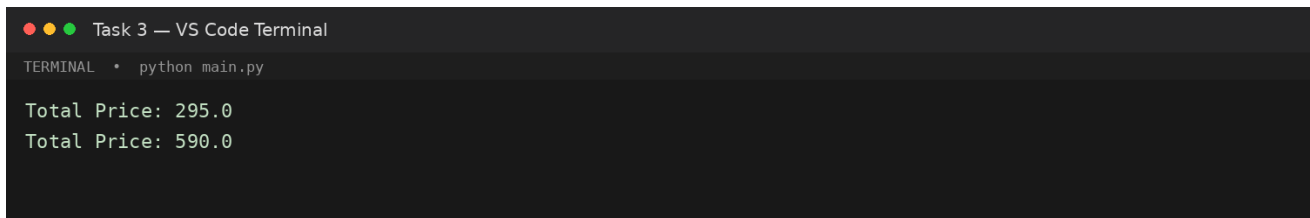
[filecite?url=0L84-L102](#)

CODE — VS CODE SCREENSHOT



```
Task 3 — VS Code
main.py
1 TAX_RATE = 0.18
2
3
4 def calculate_total(price):
5     """Calculate the total price after applying tax."""
6     tax = price * TAX_RATE
7     return price + tax
8
9
10 def main():
11     """Calculate totals for the required prices."""
12     for price in (250, 500):
13         print("Total Price:", calculate_total(price))
14
15
16 if __name__ == "__main__":
17     main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



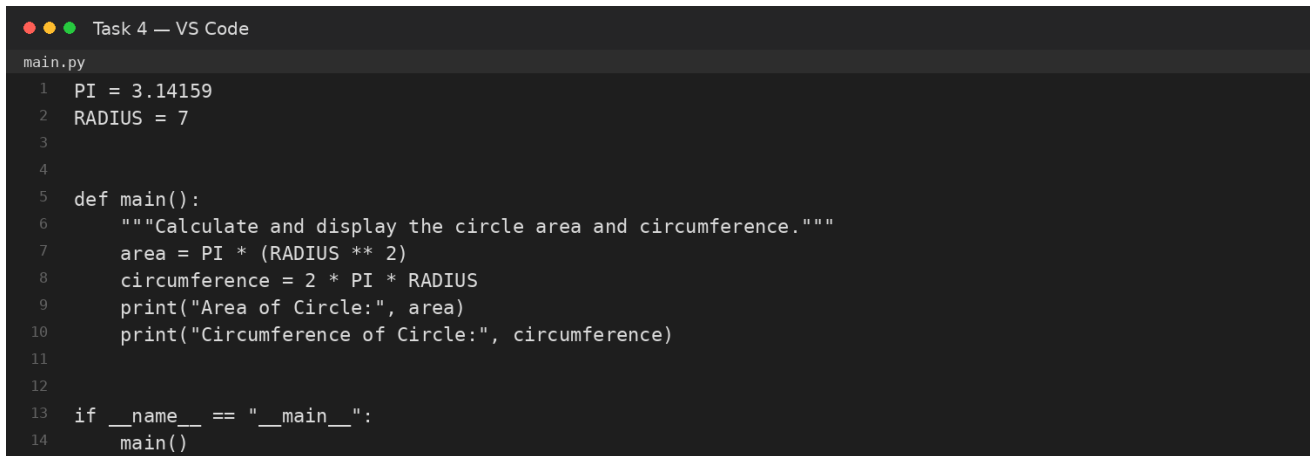
```
Task 3 — VS Code Terminal
TERMINAL • python main.py

Total Price: 295.0
Total Price: 590.0
```

4. Replacing Hardcoded Values with Constants

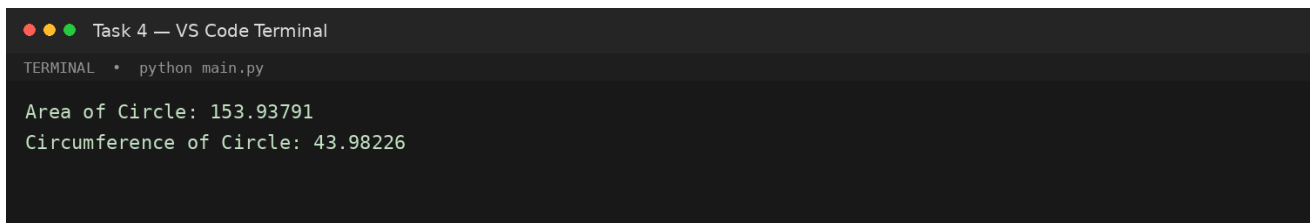
Task requirement: PI and RADIUS are named constants, making the calculations easier to understand and maintain. [filecite\turnOfile0\103-L117](#)

CODE — VS CODE SCREENSHOT



```
Task 4 — VS Code
main.py
1  PI = 3.14159
2  RADIUS = 7
3
4
5  def main():
6      """Calculate and display the circle area and circumference."""
7      area = PI * (RADIUS ** 2)
8      circumference = 2 * PI * RADIUS
9      print("Area of Circle:", area)
10     print("Circumference of Circle:", circumference)
11
12
13 if __name__ == "__main__":
14     main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



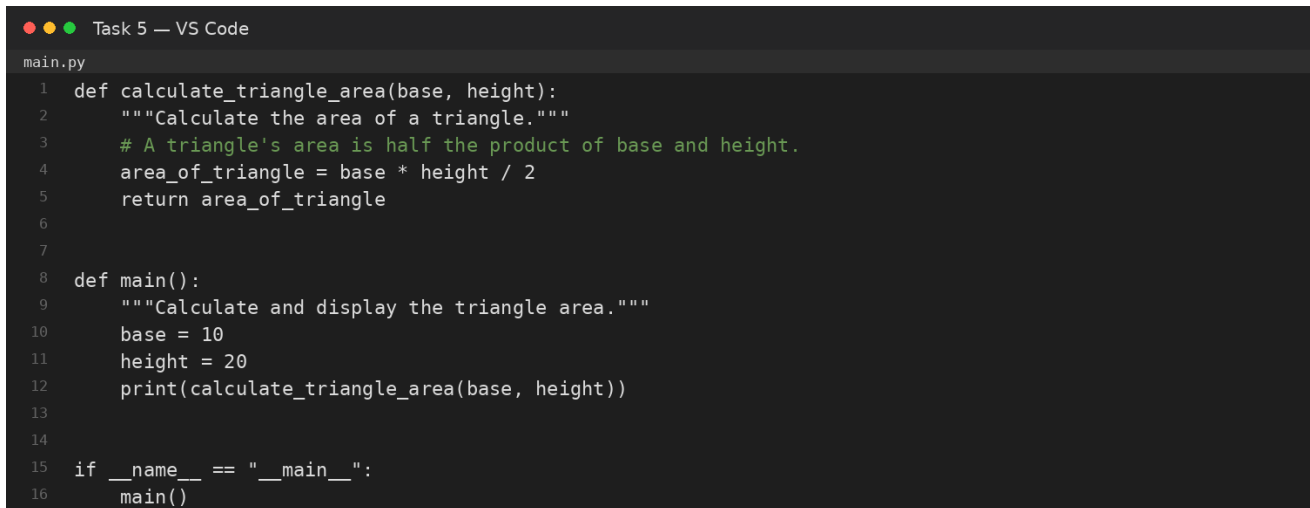
```
Task 4 — VS Code Terminal
TERMINAL • python main.py

Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

5. Improving Variable Naming and Readability

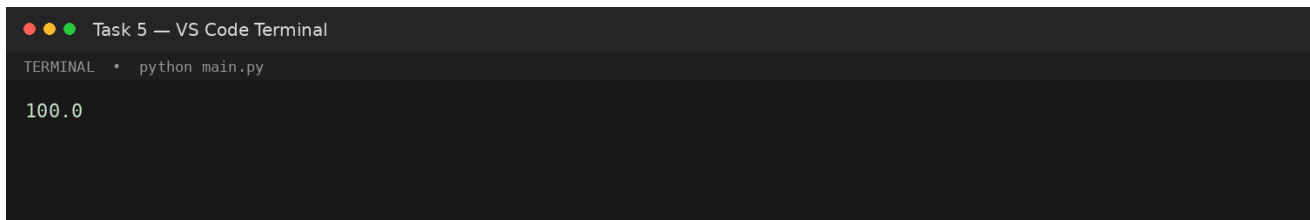
Task requirement: Unclear variables a, b, and c are replaced with descriptive names: base, height, and area_of_triangle. [filecite\turn0file0\L118-L130](#)

CODE — VS CODE SCREENSHOT



```
Task 5 — VS Code
main.py
1 def calculate_triangle_area(base, height):
2     """Calculate the area of a triangle."""
3     # A triangle's area is half the product of base and height.
4     area_of_triangle = base * height / 2
5     return area_of_triangle
6
7
8 def main():
9     """Calculate and display the triangle area."""
10    base = 10
11    height = 20
12    print(calculate_triangle_area(base, height))
13
14
15 if __name__ == "__main__":
16    main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



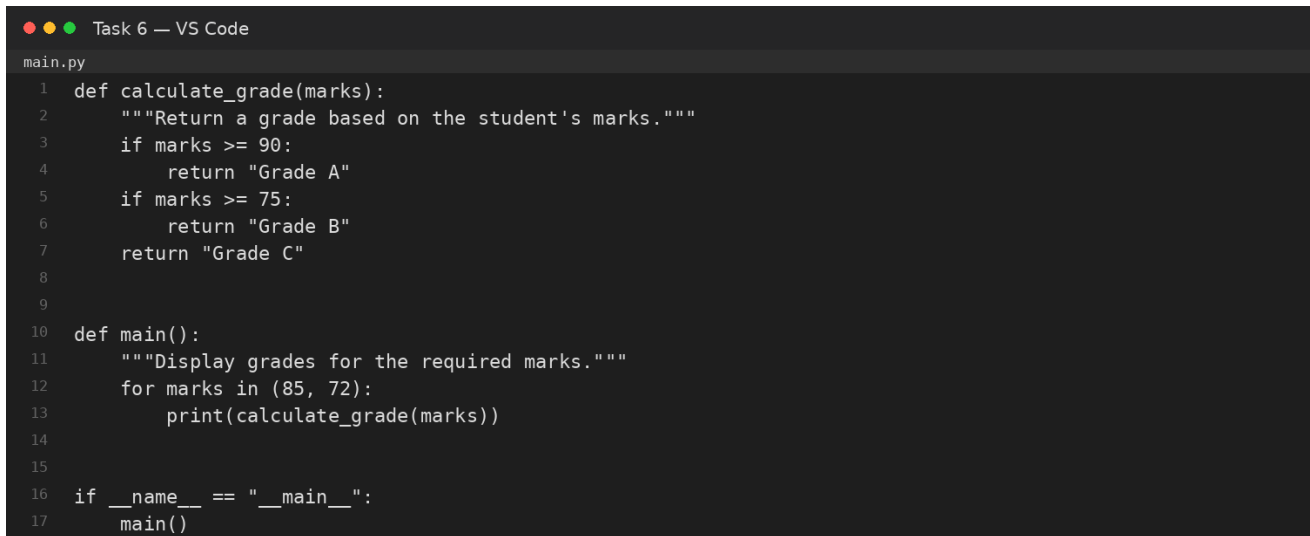
```
Task 5 — VS Code Terminal
TERMINAL • python main.py

100.0
```

6. Removing Redundant Conditional Logic

Task requirement: Repeated if-elif-else blocks are replaced with one reusable function while preserving the original grading thresholds. [filecite\turn0file0\L131-L157](#)

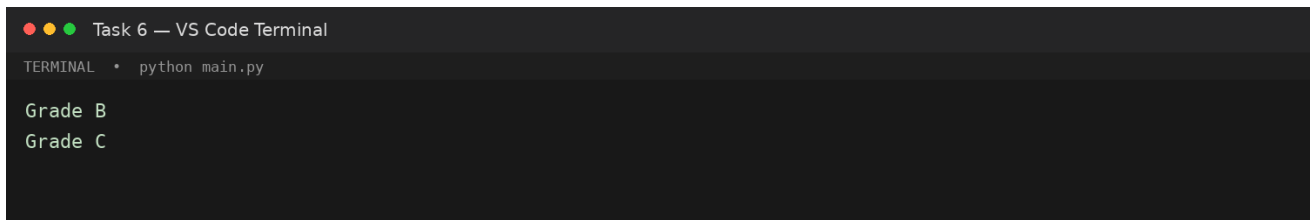
CODE — VS CODE SCREENSHOT



The screenshot shows a VS Code editor window titled "Task 6 — VS Code". The file being edited is "main.py". The code defines a function "calculate_grade(marks)" that returns "Grade A" for marks >= 90, "Grade B" for marks >= 75, and "Grade C" otherwise. A "main()" function iterates over the marks (85, 72) and prints the result of "calculate_grade(marks)". The script is executed as a main module.

```
1 def calculate_grade(marks):
2     """Return a grade based on the student's marks."""
3     if marks >= 90:
4         return "Grade A"
5     if marks >= 75:
6         return "Grade B"
7     return "Grade C"
8
9
10 def main():
11     """Display grades for the required marks."""
12     for marks in (85, 72):
13         print(calculate_grade(marks))
14
15
16 if __name__ == "__main__":
17     main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



The screenshot shows a VS Code terminal window titled "Task 6 — VS Code Terminal". The terminal shows the command "python main.py" being executed, resulting in the output "Grade B" and "Grade C" on separate lines.

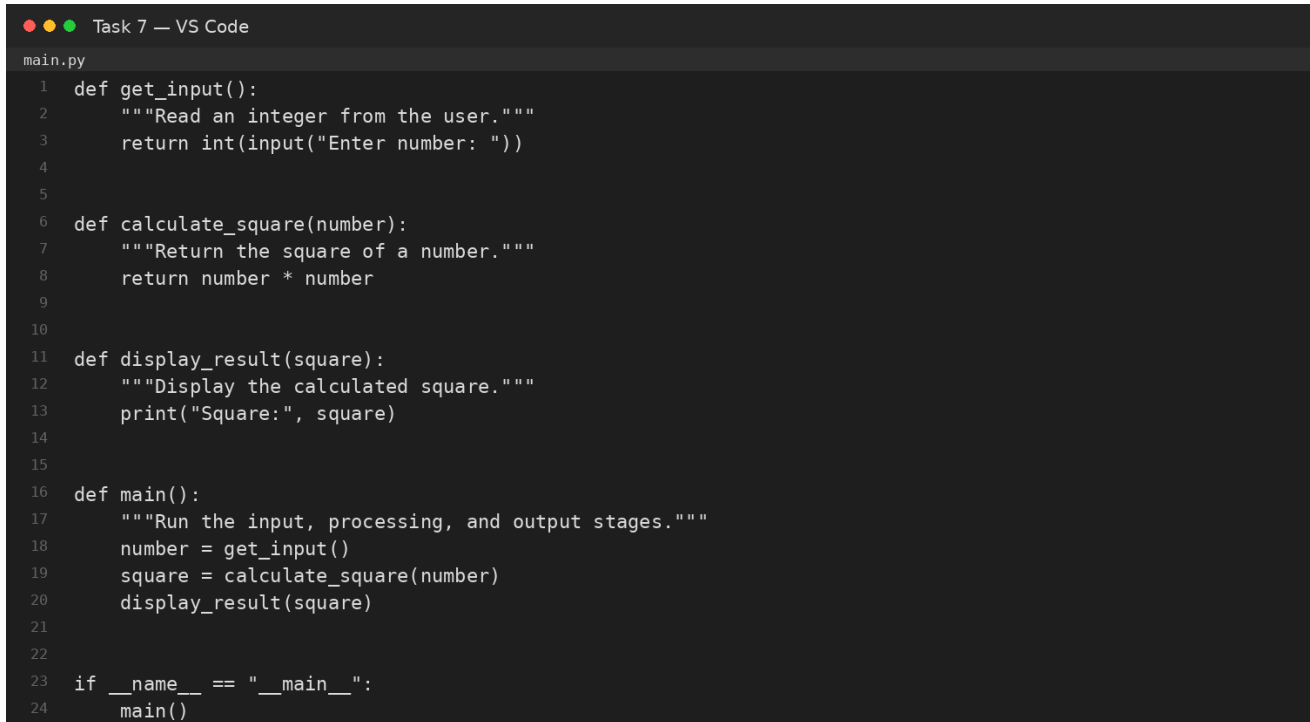
```
TERMINAL • python main.py

Grade B
Grade C
```

7. Converting Procedural Code to Functions

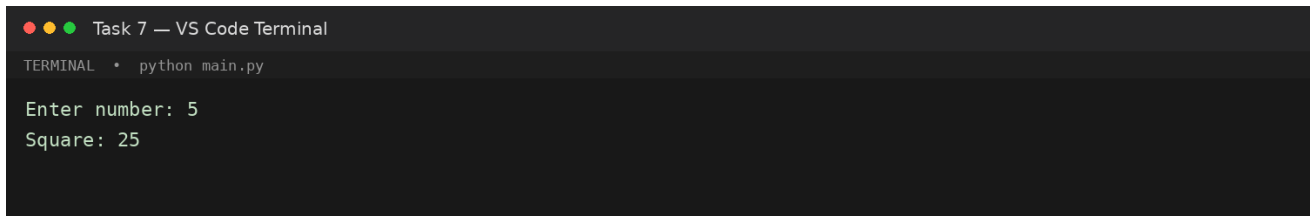
Task requirement: Input, processing, and output are separated into `get_input()`, `calculate_square()`, and `display_result()`. [filecite\turn0file0\L158-L173](#)

CODE — VS CODE SCREENSHOT



```
Task 7 — VS Code
main.py
1  def get_input():
2      """Read an integer from the user."""
3      return int(input("Enter number: "))
4
5
6  def calculate_square(number):
7      """Return the square of a number."""
8      return number * number
9
10
11 def display_result(square):
12     """Display the calculated square."""
13     print("Square:", square)
14
15
16 def main():
17     """Run the input, processing, and output stages."""
18     number = get_input()
19     square = calculate_square(number)
20     display_result(square)
21
22
23 if __name__ == "__main__":
24     main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



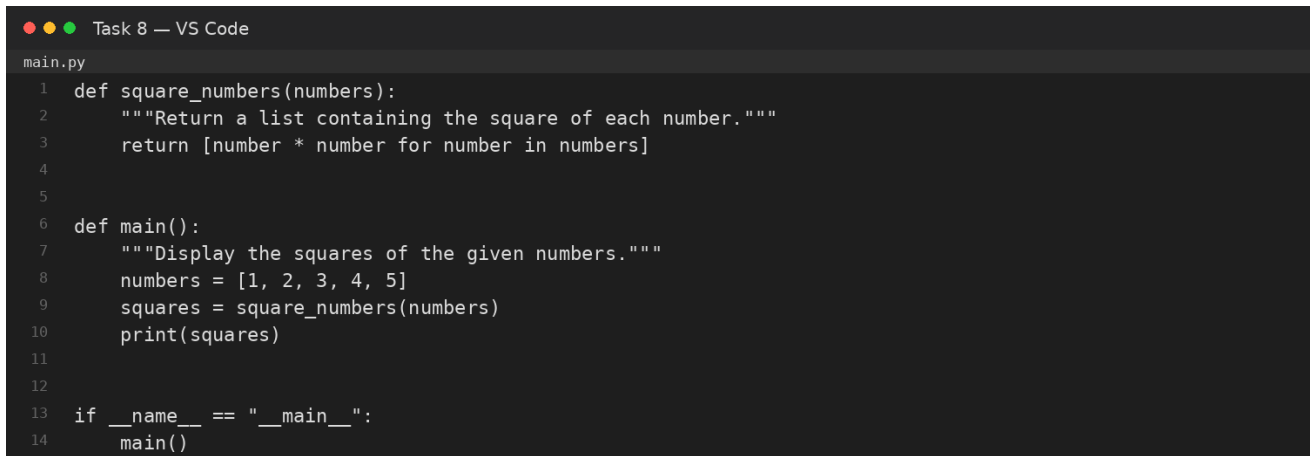
```
Task 7 — VS Code Terminal
TERMINAL • python main.py

Enter number: 5
Square: 25
```

8. Optimizing List Processing

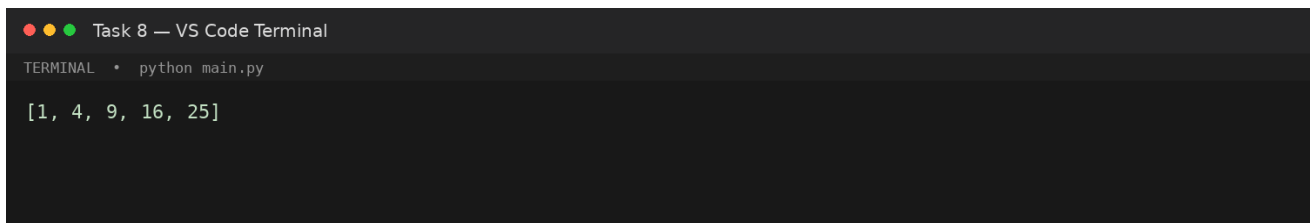
Task requirement: Manual append-based looping is replaced with a list comprehension, which is shorter and clearer while producing identical output. [filecite\turn0file0\L174-L188](#)

CODE — VS CODE SCREENSHOT



```
Task 8 — VS Code
main.py
1 def square_numbers(numbers):
2     """Return a list containing the square of each number."""
3     return [number * number for number in numbers]
4
5
6 def main():
7     """Display the squares of the given numbers."""
8     numbers = [1, 2, 3, 4, 5]
9     squares = square_numbers(numbers)
10    print(squares)
11
12
13 if __name__ == "__main__":
14    main()
```

OUTPUT — VS CODE TERMINAL SCREENSHOT



```
Task 8 — VS Code Terminal
TERMINAL • python main.py

[1, 4, 9, 16, 25]
```